

Fantasy Character Chat App

Backend Technical Architecture — MVP

Version: 1.0

Status: Technical Design

Primary Backend: Node.js + TypeScript

Database: MongoDB

Cache / Queue: Redis + BullMQ

LLM Provider: Venice AI

Architecture Style: Modular Monolith

1. Overview

The Fantasy Character Chat App is an AI-powered conversational platform where users can interact with fictional fantasy characters through persistent conversations.

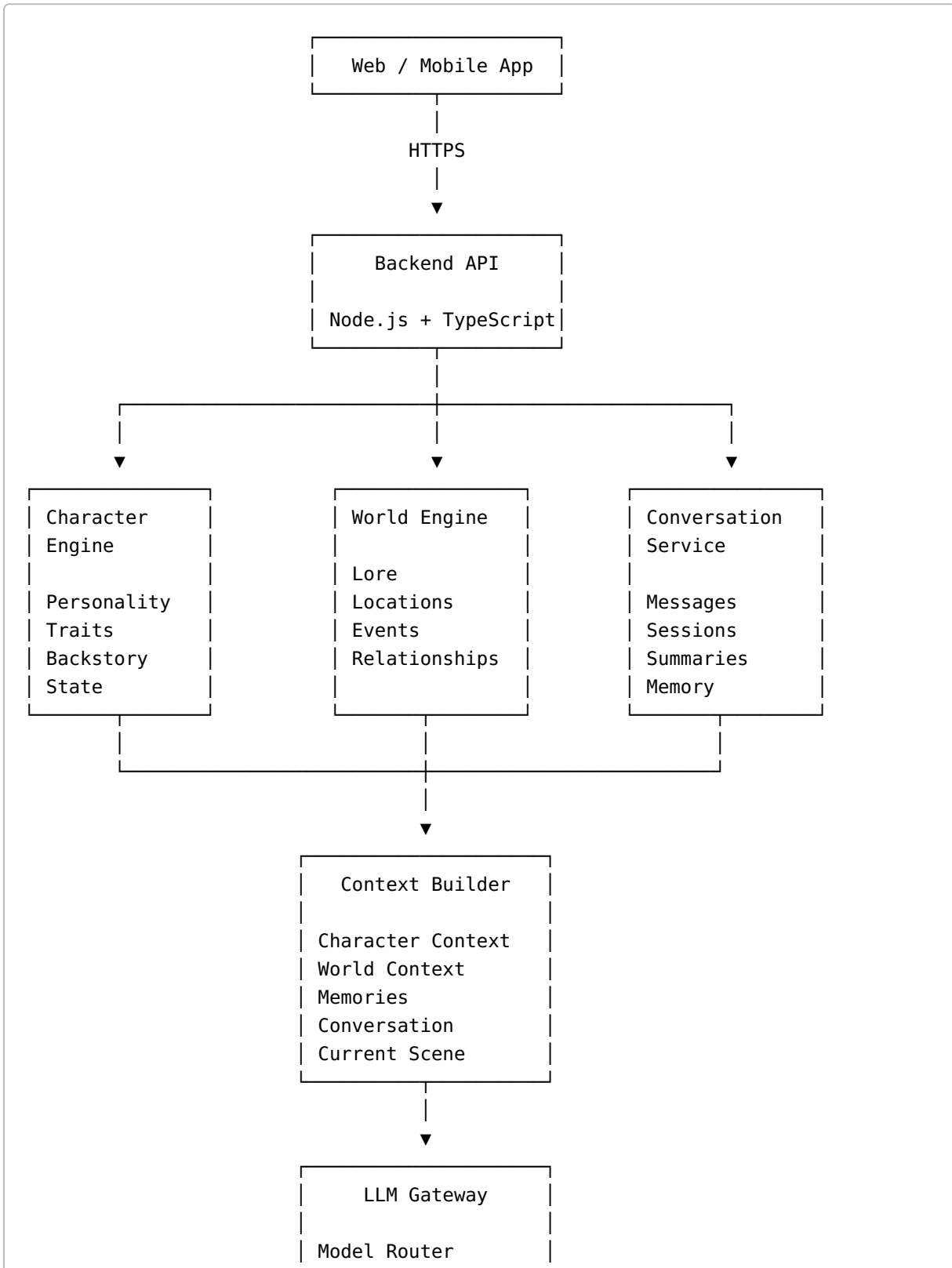
The backend is responsible for:

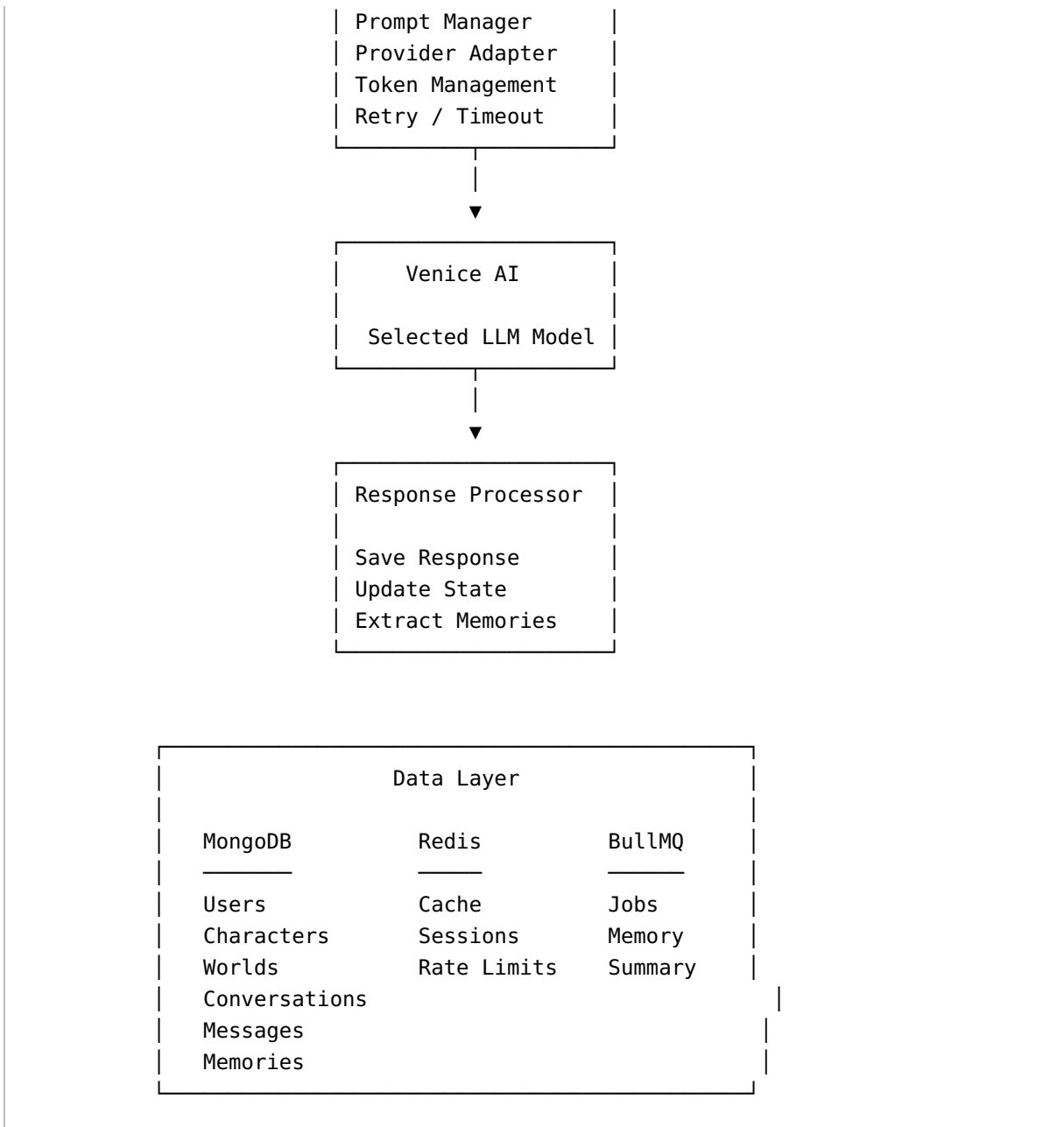
- User authentication and authorization
- Character management
- Fantasy world management
- Conversation management
- Character personality and state
- Long-term user and character memory
- Context construction
- LLM provider abstraction
- Model selection and switching
- AI response generation
- Background memory extraction
- Caching and rate limiting

For the MVP, the backend will use a **modular monolith architecture** rather than microservices.

The LLM will be accessed through the **Venice AI API**, while the application will maintain its own abstraction layer so the provider can be replaced later.

2. High-Level Architecture



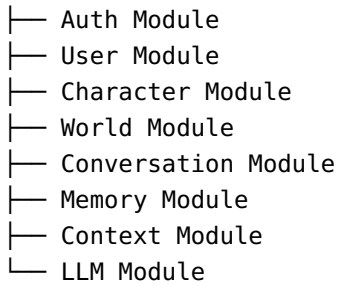


3. Architecture Style

The MVP will use a **Modular Monolith**.

All core business modules will initially run inside one Node.js application.

Node.js Application



- └─ Auth Module
- └─ User Module
- └─ Character Module
- └─ World Module
- └─ Conversation Module
- └─ Memory Module
- └─ Context Module
- └─ LLM Module

This approach provides clear separation between domains without introducing the operational complexity of microservices.

The architecture should nevertheless maintain clear module boundaries so individual modules can be extracted into independent services later if necessary.

4. Technology Stack

Backend

- Node.js
- TypeScript
- Express.js or Fastify
- Mongoose
- MongoDB

AI

- Venice AI API
- OpenAI-compatible API interface where applicable
- Configurable model selection

Caching

- Redis

Background Processing

- BullMQ
- Redis

Authentication

Recommended:

- JWT access tokens
- Refresh tokens
- Password hashing using Argon2 or bcrypt

Validation

Recommended:

- Zod

Logging

Recommended:

- Pino
-

5. Core Backend Modules

The backend will contain the following modules:

```
src/
├── modules/
│   ├── auth/
│   ├── users/
│   ├── characters/
│   ├── worlds/
│   ├── conversations/
│   ├── memories/
│   ├── context/
│   └── llm/
├── infrastructure/
│   ├── database/
│   ├── redis/
│   ├── queue/
│   └── logging/
└── middleware/
```

```
|
|— config/
|
|— app.ts
```

6. Character Engine

The Character Engine manages the identity and behavior of AI characters.

A character should contain significantly more information than just a name and personality description.

Character Components

```
Character
|
|— Basic Information
|   |— Name
|   |— Description
|   |— Avatar
|   └— Age / Character metadata
|
|— Personality
|   |— Traits
|   |— Temperament
|   |— Values
|   └— Behavioral patterns
|
|— Communication Style
|   |— Formality
|   |— Sentence length
|   |— Vocabulary
|   └— Emoji usage
|
|— Backstory
|
|— Relationships
|
|— Speech Patterns
|
└— Current State
    |— Mood
    └— Trust
```

- └ Affection
- └ Other dynamic attributes

Example

```
{
  "name": "Luna",
  "description": "A mysterious mage from the northern kingdom.",
  "personality": {
    "traits": [
      "confident",
      "playful",
      "mysterious"
    ],
    "temperament": "playful"
  },
  "communicationStyle": {
    "formality": "casual",
    "sentenceLength": "short",
    "emojiUsage": "low"
  },
  "backstory": "...",
  "currentState": {
    "mood": "curious",
    "trust": 0.72,
    "affection": 0.81
  }
}
```

The Character Engine converts this information into structured context that can be consumed by the Context Builder.

7. World Engine

The World Engine manages the fictional universe.

It prevents the AI from having to recreate the entire fantasy world from scratch on every request.

World Components

```
World
|
```

```
├─ Kingdoms
├─ Cities
├─ Locations
├─ Factions
├─ Characters
├─ History
├─ Lore
├─ Magic System
├─ Events
└─ Relationships
```

For example:

```
Eldoria
|
├─ Northern Castle
|   ├── Ruler: Queen Aria
|   ├── Population
|   └─ Important locations
|
├─ Silver Forest
|
├─ Dark Mountains
|
└─ Royal City
```

The World Engine provides only the relevant world information to the Context Builder.

8. Conversation Service

The Conversation Service manages user conversations with characters.

A conversation contains:

```
Conversation
|
├─ User
├─ Character
├─ World
├─ Messages
└─ Conversation Summary
```

```
|— Current Scene
|— Conversation Metadata
```

The system should not send the entire historical conversation to the LLM indefinitely.

Instead, the Context Builder should combine:

```
Recent Messages
+
Conversation Summary
+
Important Memories
+
Current Character State
+
Current World State
```

This keeps context manageable and reduces LLM costs.

9. Memory System

Memory is critical for making characters feel persistent.

The system should distinguish between temporary conversation information and long-term memories.

Memory Types

```
Memory
|
|— User Preference
|— Important Event
|— Relationship Event
|— Character Knowledge
|— World Event
|— Conversation Fact
```

Example:

```
{
  "userId": "...",
  "characterId": "...",
```

```
"type": "user_preference",  
"content": "The user likes silver swords.",  
"importance": 0.8  
}
```

During future conversations, relevant memories can be retrieved and injected into the context.

10. MongoDB Data Model

The MVP will use MongoDB as the primary database.

Recommended collections:

```
users  
characters  
worlds  
conversations  
messages  
memories  
conversation_summaries  
model_configs
```

Users

```
users  
├─ _id  
├─ email  
├─ username  
├─ passwordHash  
├─ profile  
├─ createdAt  
└─ updatedAt
```

Characters

```
characters  
├─ _id  
├─ ownerId  
├─ name  
├─ description  
└─ personality
```

```
|— backstory
|— communicationStyle
|— relationships
|— currentState
|— worldId
|— visibility
|— createdAt
└— updatedAt
```

Worlds

```
worlds
|— _id
|— ownerId
|— name
|— description
|— lore
|— locations
|— factions
|— events
└— createdAt
```

Conversations

```
conversations
|— _id
|— userId
|— characterId
|— worldId
|— title
|— currentScene
|— summary
|— metadata
|— createdAt
└— updatedAt
```

Messages

```
messages
|— _id
|— conversationId
|— role
|— content
```

```
├─ tokenUsage
├─ model
├─ metadata
└─ createdAt
```

Roles:

```
user
assistant
system
```

Memories

```
memories
├─ _id
├─ userId
├─ characterId
├─ conversationId
├─ type
├─ content
├─ importance
├─ metadata
├─ createdAt
└─ updatedAt
```

11. Context Builder

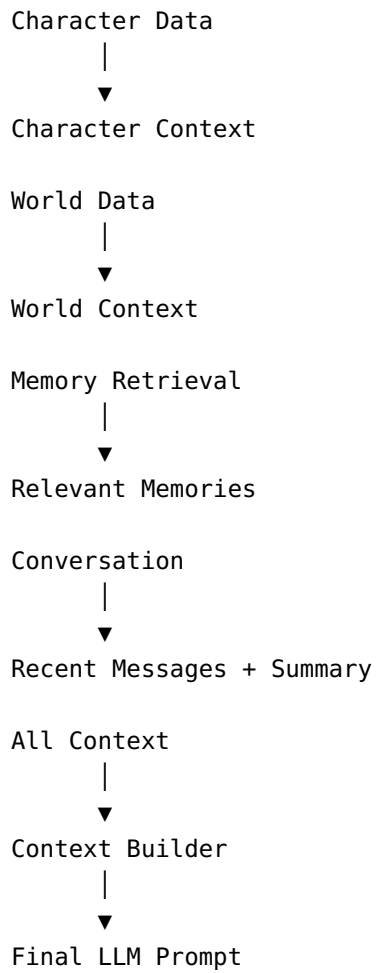
The Context Builder is responsible for constructing the final LLM request.

It should not blindly concatenate all available data.

Input

```
Character
World
User
Conversation
Memories
Current Scene
Current User Message
```

Processing



12. Context Structure

A generated context can conceptually look like:

SYSTEM CONTEXT

CHARACTER

Name: Luna

PERSONALITY

- Confident
- Playful
- Mysterious

BACKSTORY

...

COMMUNICATION STYLE

...

WORLD

Current world: Eldoria

Current location: Northern Castle

CURRENT SCENE

...

RELATIONSHIP STATE

Trust: 72%

Affection: 81%

RELEVANT MEMORIES

...

CONVERSATION SUMMARY

...

RECENT MESSAGES

...

USER MESSAGE

...

The Context Builder should have token-budget rules so that context does not grow without limits.

13. LLM Gateway

The LLM Gateway provides an abstraction between the application and external AI providers.

The Character Engine and other business modules should **never directly call Venice**.

Incorrect:

Character Service

|

▼
Venice API

Correct:

```
Character Engine
  |
  ▼
Context Builder
  |
  ▼
LLM Gateway
  |
  ▼
Model Router
  |
  ▼
Venice Provider
  |
  ▼
Venice API
```

This allows the AI provider to be changed later.

14. LLM Provider Interface

The backend should define a generic interface:

```
interface LLMPProvider {
    generate(
        request: LLMRequest
    ): Promise<LLMResponse>;
}
```

The Venice implementation:

```
class VeniceProvider implements LLMPProvider {
    async generate(
        request: LLMRequest
    ): Promise<LLMResponse> {
        // Call Venice API
    }
}
```

```
}  
}
```

Future providers can implement the same interface:

```
LLMProvider  
|  
├── VeniceProvider  
├── OpenRouterProvider  
├── HuggingFaceProvider  
└── OtherProvider
```

This prevents vendor lock-in.

15. Model Router

The Model Router determines which model should process a request.

For the MVP, it can be simple:

```
Request  
|  
▼  
Model Router  
|  
▼  
Venice  
|  
▼  
Configured Model
```

The system should nevertheless store model configuration separately.

Example:

```
{  
  "provider": "venice",  
  "model": "venice-uncensored",  
  "enabled": true,  
  "priority": 1,  
}
```

```
"maxTokens": 1000
}
```

Later, multiple models can be configured:

```
Model Router
|
├─ Model A → Venice
├─ Model B → Venice
├─ Model C → OpenRouter
└─ Model D → Hugging Face
```

16. Prompt Manager

Prompts should not be hardcoded throughout the application.

Create a dedicated Prompt Manager.

```
Prompt Manager
|
├─ Character Prompt
├─ World Prompt
├─ Conversation Prompt
├─ Memory Prompt
└─ System Prompt
```

Example:

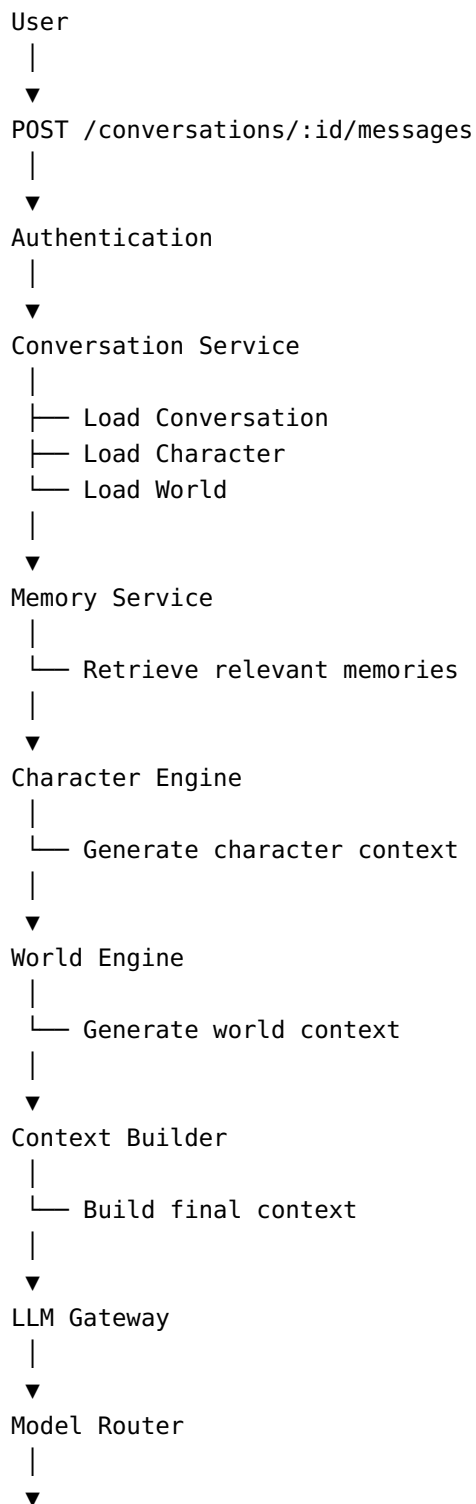
```
interface PromptContext {
    character: CharacterContext;
    world: WorldContext;
    memories: MemoryContext[];
    conversation: ConversationContext;
    userMessage: string;
}
```

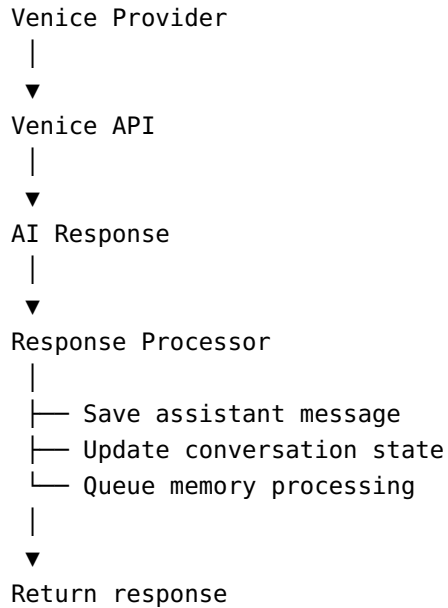
The Prompt Manager converts this into the final LLM request.

This also makes prompt versioning possible later.

17. Complete Message Flow

When the user sends a message:

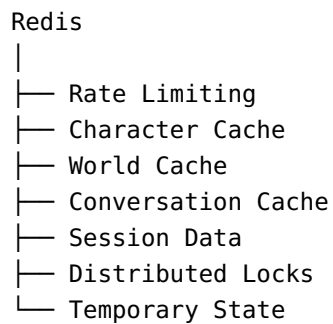




18. Redis

Redis will be used for temporary and high-frequency data.

Recommended use cases:



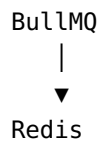
MongoDB remains the source of truth for persistent data.

Redis should not be treated as the permanent database.

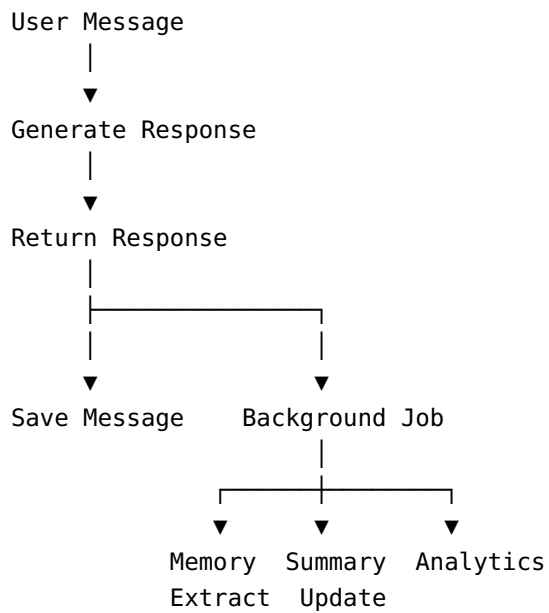
19. Background Jobs

Some operations should not block the user's response.

Use:



Example:



Potential jobs:

```
memory.extract
conversation.summarize
character.state.update
analytics.process
```

The MVP does not need all of these immediately.

20. API Design

Authentication

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

Characters

```
GET /api/v1/characters
POST /api/v1/characters
GET /api/v1/characters/:id
PATCH /api/v1/characters/:id
DELETE /api/v1/characters/:id
```

Worlds

```
GET /api/v1/worlds
POST /api/v1/worlds
GET /api/v1/worlds/:id
PATCH /api/v1/worlds/:id
DELETE /api/v1/worlds/:id
```

Conversations

```
POST /api/v1/conversations
GET /api/v1/conversations
GET /api/v1/conversations/:id
DELETE /api/v1/conversations/:id
```

Messages

```
POST /api/v1/conversations/:id/messages
GET /api/v1/conversations/:id/messages
```

The primary AI endpoint is:

```
POST /api/v1/conversations/:id/messages
```

Request:

```
{
  "message": "What happens next?"
}
```

Response:

```
{
  "message": {
    "id": "...",
    "role": "assistant",
    "content": "..."
  },
  "usage": {
    "inputTokens": 1200,
    "outputTokens": 350
  }
}
```

21. Recommended Backend Directory Structure

```
src/
|
├─ app.ts
├─ server.ts
|
├─ config/
|   ├─ env.ts
|   └─ constants.ts
|
├─ modules/
|   |
|   ├─ auth/
|   |   ├─ auth.controller.ts
|   |   ├─ auth.service.ts
|   |   ├─ auth.repository.ts
|   |   ├─ auth.routes.ts
|   |   └─ auth.schema.ts
```



```
| | | workers/
| | | logger/
| | |
| | | middleware/
| | | | auth.middleware.ts
| | | | error.middleware.ts
| | | | rate-limit.middleware.ts
| | |
| | | utils/
```

22. Error Handling

The LLM Gateway should handle external provider failures.

Example:

```
Venice Request
|
| | Success
| |   ↓
| | Response
| |
| | Failure
| |   |
| |   | Timeout
| |   | Rate Limit
| |   | Server Error
| |   | Invalid Request
```

The gateway should implement:

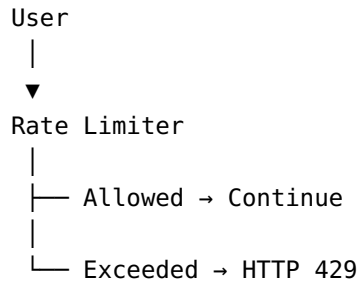
- Request timeout
- Retry for appropriate transient errors
- Exponential backoff
- Provider error normalization
- Logging
- Usage tracking

The API should never expose raw Venice errors directly to clients.

23. Rate Limiting

Rate limiting should be implemented at the API level.

Example:



Different limits can eventually be applied to:

```
Free User
Premium User
Admin
Internal API
```

AI generation should have a separate rate limit from ordinary API requests because LLM requests directly generate external costs.

24. Cost Management

Because the application uses a paid external LLM API, token usage must be tracked.

Each AI request should record:

```
inputTokens
outputTokens
totalTokens
model
provider
latency
conversationId
userId
```

Example:

```
{
  "provider": "venice",
  "model": "venice-uncensored",
  "inputTokens": 1800,
  "outputTokens": 420,
  "totalTokens": 2220,
  "latencyMs": 3200
}
```

This allows future dashboards for:

- Cost per user
- Cost per conversation
- Cost per message
- Daily LLM cost
- Monthly LLM cost
- Model usage
- Average token usage

25. Security

The backend must keep the Venice API key server-side.

Never expose:

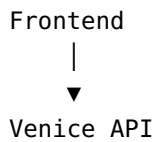
```
VENICE_API_KEY
```

to the frontend.

The architecture must be:

```
Frontend
  |
  ▼
Your Backend
  |
  ▼
Venice API
```

Never:



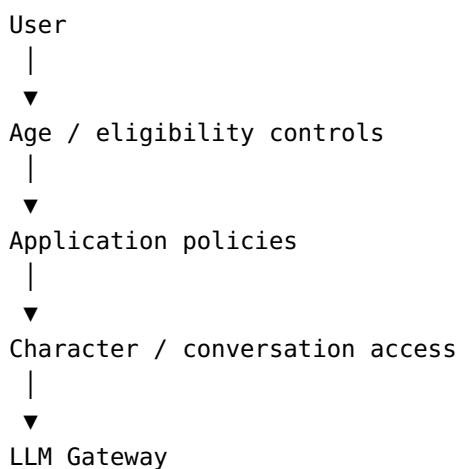
Other security requirements:

- Password hashing
- JWT validation
- Request validation
- MongoDB query sanitization
- Rate limiting
- Secure HTTP headers
- Environment-based secrets
- Input size limits
- Authorization checks
- Ownership validation for characters/worlds/conversations
- Audit logging for sensitive administrative operations

26. Important Product Safety Controls

Because the application is intended for adult fantasy roleplay, the application layer should not rely exclusively on the LLM provider for user protection.

The platform should implement:



The exact policies should be defined according to the jurisdictions and app stores/payment providers where the product will operate.

The backend should also maintain clear separation between:

```
User content
Character configuration
World/lore data
LLM-generated content
Platform policy
```

This makes future moderation and policy enforcement substantially easier.

27. Initial MVP Scope

The first version should **not** attempt to build every planned subsystem.

Phase 1

Implement:

```
Authentication
  ↓
Character CRUD
  ↓
World CRUD
  ↓
Conversation CRUD
  ↓
Message generation
  ↓
Context Builder
  ↓
LLM Gateway
  ↓
Venice
  ↓
MongoDB
```

Phase 2

Add:

```
Redis
  ↓
```

Caching + Rate limiting

BullMQ



Memory extraction



Conversation summarization

Phase 3

Add:

Advanced Memory

Character State

World State

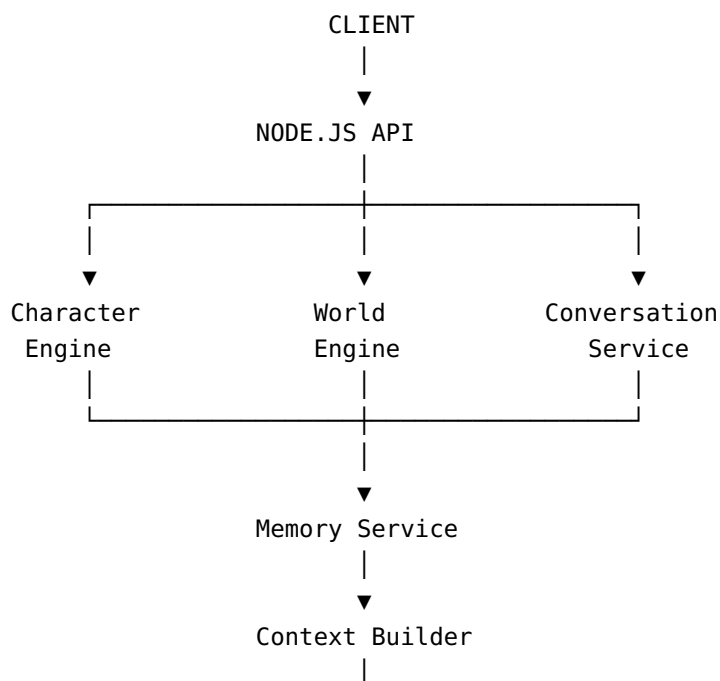
Model Router

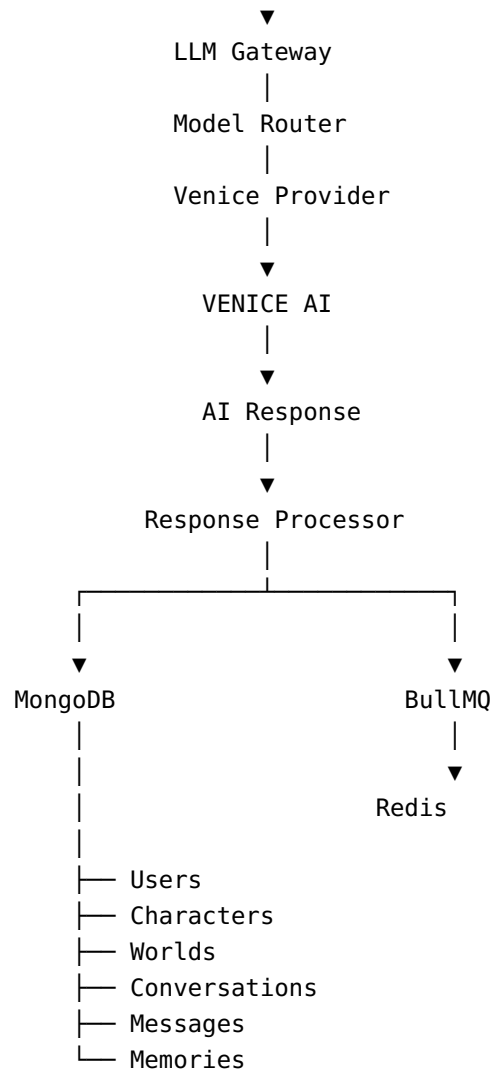
Usage Analytics

Multiple LLM Providers

28. Final MVP Architecture

The recommended initial implementation is:





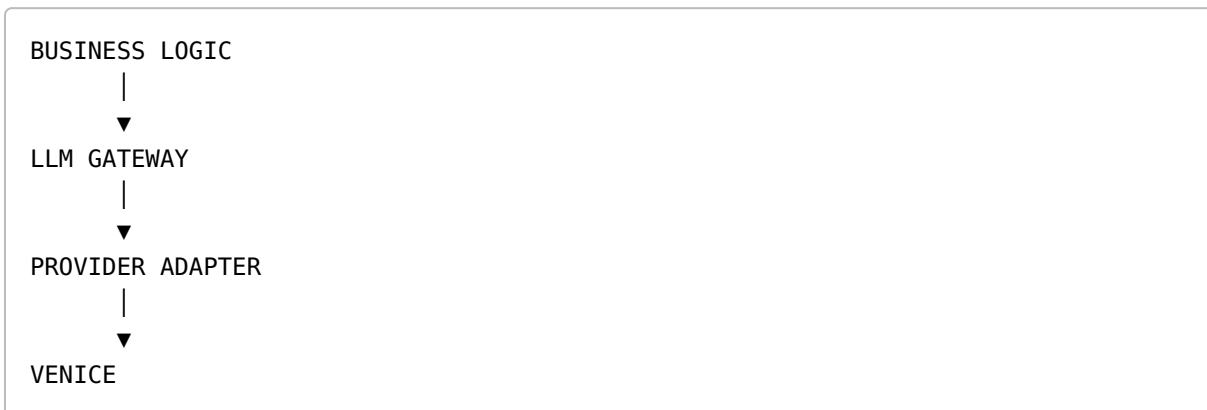
29. Key Architectural Decisions

Decision	MVP Choice
Backend	Node.js + TypeScript
Architecture	Modular Monolith
Database	MongoDB
ODM	Mongoose
Cache	Redis
Background Jobs	BullMQ

Decision	MVP Choice
LLM Provider	Venice AI
LLM Abstraction	LLM Gateway
Model Switching	Model Router
Context Management	Dedicated Context Builder
Character Logic	Character Engine
World Logic	World Engine
Memory	MongoDB + background processing
API	REST
Authentication	JWT
Validation	Zod
Logging	Pino
Streaming	Excluded from MVP
Load Balancing	Excluded from MVP
Production Deployment Design	Excluded from MVP

30. Core Principle

The most important architectural rule is:



Never couple the Character Engine, World Engine, or Conversation Service directly to Venice.

The application should treat the LLM as an interchangeable infrastructure dependency.

That means the future migration can be:

Today:

Your Backend

↓

LLM Gateway

↓

Venice

Future:

Your Backend

↓

LLM Gateway

↓

Model Router

├ Venice

├ Hugging Face

├ OpenRouter

└ Self-hosted model

The Character Engine, World Engine, Memory System, MongoDB schema, API contracts, and frontend can remain largely unchanged.

This gives the MVP a simple implementation path while keeping the core architecture flexible enough for the product to evolve.